UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Actividad Parcial 2

# Búsqueda local y optimización

Sección 3 · Tema 2.3 · Implementación + Optimización

| | |
|---|---|
| **Alumno(a):** | SANVICENTE VALENCIA FERNANDO YAIR |
| **Programa:** | Ing. en Tecnologías de la Información e Innovación Digital |
| **Asignatura:** | Fundamentos de Inteligencia Artificial |
| **Grupo:** | 171 · Cuatrimestre 7 |
| **Docente:** | Mtro. Javier García Lara |
| **Tipo de entrega:** | Individual |
| **Fecha límite de entrega:** | 22 de octubre de 2026 |

**Archivos de la entrega:** este PDF y el script `FIA_6171_Sanvicente_S3.py` (código completo en la sección 7).

## 1. Enunciado y sección

**Sección:** 3 de 6 (50 pts, individual). **Tema:** 2.3 Búsqueda local. Se implementan en Python los algoritmos **Hill Climbing** y **Simulated Annealing** para optimizar una función de costo, se reportan la función objetivo, la vecindad y los parámetros, y se comparan los resultados con gráficas de convergencia.

**Enunciado del problema.** Un repartidor debe visitar 30 puntos de entrega exactamente una vez y regresar al punto de partida. Se busca el recorrido de menor longitud total (problema del agente viajero, TSP, en su versión simétrica y euclidiana).

## 2. Datos del problema

- **Puntos:** 30 coordenadas aleatorias en un plano de 100 × 100, generadas con semilla fija (2026) para que el experimento sea reproducible. Son datos sintéticos, no de una ciudad real.
- **Distancia:** euclidiana entre cada par de puntos (matriz de 30 × 30).
- **Espacio de búsqueda:** $(30 - 1)! / 2 \approx 4.4 \times 10^{30}$ recorridos distintos, por lo que revisarlos todos es imposible.
- **Instancia de validación:** una versión de 15 puntos, donde sí se puede calcular el óptimo exacto con programación dinámica (Held-Karp), para comprobar que los algoritmos funcionan bien.

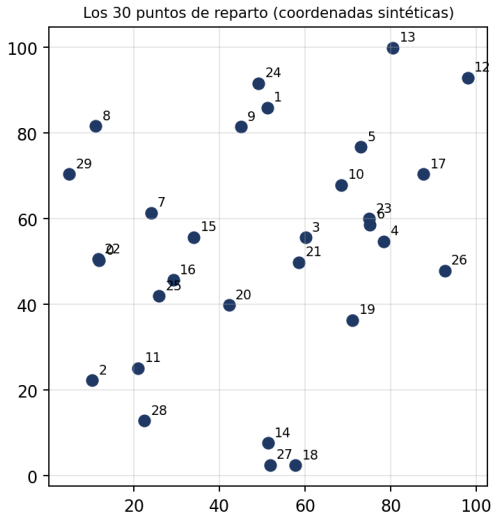



Figura 1. Los 30 puntos de entrega (el punto 0 es el de partida). Elaboración propia.

## 3. Formulación

| Elemento | Definición |
|---|---|
| Estado | Una permutación de los 30 puntos: el orden en que se visitan (el circuito se cierra al volver al primero). |
| Función objetivo | f(ruta) = suma de las distancias entre puntos consecutivos, incluyendo el regreso al inicio. Se **minimiza**. |
| Vecindad | **2-opt**: se eligen dos posiciones i < j al azar y se invierte el tramo entre ellas. Reemplaza dos aristas por otras dos y siempre da una ruta válida. Hay hasta C(30, 2) = 435 vecinos por estado. |
| Estado inicial | Permutación aleatoria (la misma para ambos algoritmos en cada corrida). |
| Criterio de paro | Presupuesto fijo de 20,000 iteraciones (evaluaciones de un vecino) para cada algoritmo. |

## 4. Procedimiento y parámetros

**Hill Climbing.** En cada iteración genera un vecino al azar y se mueve a él solo si su costo es menor. Nunca empeora, así que puede quedar atrapado en un óptimo local.

**Simulated Annealing.** Genera un vecino y, si mejora, se mueve a él. Si empeora en $\Delta$, se mueve igualmente con probabilidad $e^{-\Delta/T}$. La temperatura T baja en cada iteración como $T \leftarrow \eta \cdot T$ (enfriamiento geométrico). Al inicio (T alta) acepta muchos empeoramientos y explora; al final (T baja) se comporta casi como Hill Climbing. Se guarda la mejor ruta encontrada.

| Parámetro | Valor | Aplica a |
|---|---|---|
| Iteraciones | 20,000 | Ambos |
| Corridas independientes | 30 (semillas 0 a 29; el mismo estado inicial para ambos en cada corrida) | Ambos |
| Temperatura inicial $T_0$ | 50 | Simulated Annealing |
| Factor de enfriamiento $\eta$ | 0.9995 (T final $\approx$ 0.0023) | Simulated Annealing |
| Vecindad | 2-opt | Ambos |

Se usó el mismo presupuesto de iteraciones, la misma vecindad y los mismos estados iniciales para que la comparación sea justa. Como los algoritmos son aleatorios, se repitió cada uno 30 veces y se reportan estadísticas.

# 5. Solución y resultados

## 5.1 Validación con 15 puntos (óptimo exacto conocido)

Con 15 puntos el óptimo exacto, calculado con Held-Karp, es 330.67. Los resultados de 30 corridas fueron:

| Algoritmo | Mejor | Media | Desv. est. | Peor | Veces que halló el óptimo |
|---|---|---|---|---|---|
| Hill Climbing | 330.67 | 341.10 | 17.35 | 374.56 | 22 de 30 |
| Simulated Annealing | 330.67 | 330.67 | 0.00 | 330.67 | 30 de 30 |

Ambos algoritmos son capaces de encontrar el óptimo, lo que valida la implementación. Hill Climbing quedó atrapado en óptimos locales en 8 de 30 corridas, mientras que Simulated Annealing llegó al óptimo en todas.

## 5.2 Instancia principal con 30 puntos

Aquí ya no se puede calcular el óptimo exacto, así que se usa como referencia el **mejor costo conocido**: 455.16, el menor encontrado entre todas las corridas del experimento. No está demostrado que sea el óptimo. Como línea base, la heurística del vecino más cercano da 552.46.

| Algoritmo | Mejor | Media | Desv. est. | Peor | Brecha media vs. referencia |
|---|---|---|---|---|---|
| Vecino más cercano (línea base) | 552.46 | — | — | — | 21.4 % |
| Hill Climbing | 461.49 | 479.55 | 11.97 | 510.07 | 5.36 % |
| Simulated Annealing | 455.16 | 466.89 | 7.14 | 481.66 | 2.58 % |

Simulated Annealing tuvo mejor resultado que Hill Climbing en 24 de las 30 corridas pareadas (mismo estado inicial) y HC fue mejor en las 6 restantes. Frente al vecino más cercano, ambos acortan la ruta: Hill Climbing en promedio 13.2 % y Simulated Annealing 15.5 %.

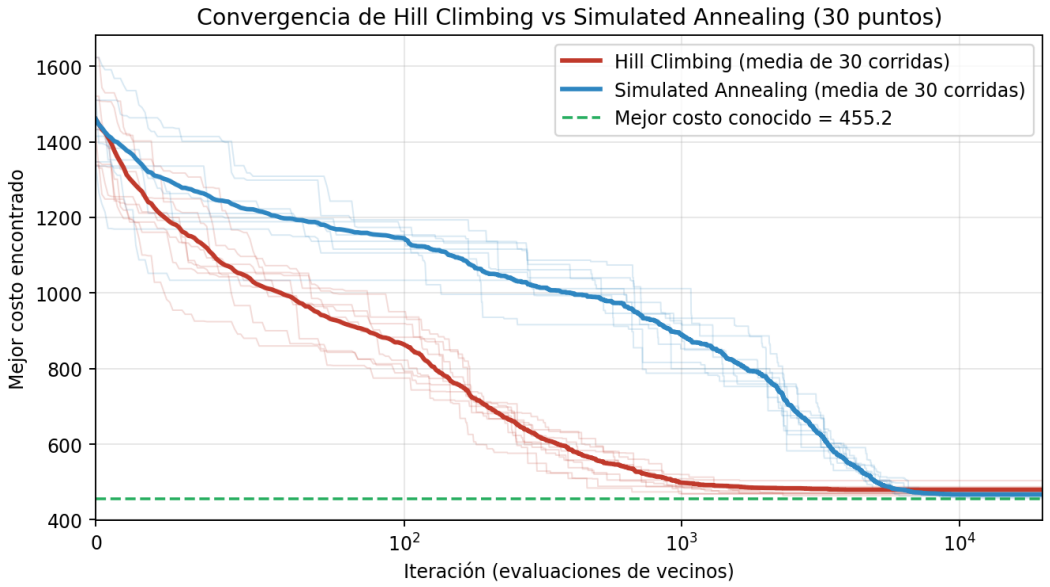



Figura 2. Gráfica de convergencia: mejor costo encontrado en cada iteración. Las líneas gruesas son la media de 30 corridas y las tenues, ocho corridas individuales (eje X en escala logarítmica). Elaboración propia.

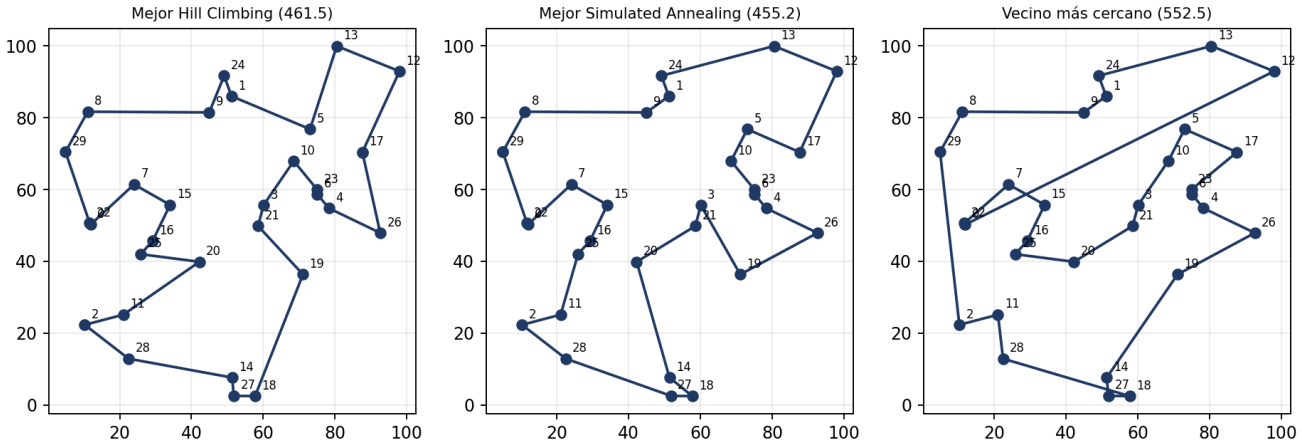



Figura 3. Mejor ruta de cada algoritmo y ruta del vecino más cercano. Elaboración propia.

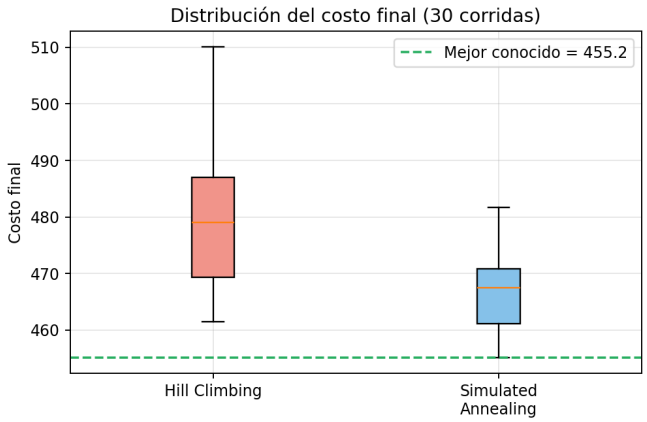



Figura 4. Distribución del costo final en las 30 corridas. Elaboración propia.

## 5.3 Sensibilidad de los parámetros de Simulated Annealing

| $T_0$ | η | T final | Costo medio | Mejor | Desv. est. |
|---|---|---|---|---|---|
| 10 | 0.9995 | 0.0005 | 466.27 | 455.16 | 9.24 |
| 50 (usado) | 0.9995 | 0.0023 | 466.89 | 455.16 | 7.14 |
| 200 | 0.9995 | 0.0091 | 464.76 | 455.48 | 8.39 |
| 50 | 0.999 (enfría rápido) | ≈ 0 | 467.48 | 455.16 | 10.68 |
| 50 | 0.9999 (enfría lento) | 6.77 | 508.58 | 475.46 | 14.60 |

Con η = 0.9995, cambiar $T_0$ entre 10 y 200 casi no modificó el resultado. Con η = 0.9999 el resultado empeoró mucho: con 20,000 iteraciones la temperatura todavía era 6.77 al final, es decir, el algoritmo seguía aceptando empeoramientos y no alcanzó a «asentarse». Con η = 0.999 enfrió más rápido, con resultados parecidos pero más dispersos.

## 6. Análisis de resultados

- **Velocidad de convergencia:** Hill Climbing baja más rápido al inicio y se estanca alrededor de unas 2,000 iteraciones (cerca de 480 de costo medio). Simulated Annealing baja más lento porque al inicio acepta rutas peores, pero sigue mejorando hasta cerca de las 8,000 iteraciones y termina en un costo medio menor (466.89).
- **Calidad y consistencia:** Simulated Annealing obtuvo menor costo medio, menor desviación (7.14 frente a 11.97) y menor brecha respecto a la referencia (2.58 % frente a 5.36 %). En la instancia de 15 puntos halló el óptimo en 30 de 30 corridas y Hill Climbing en 22 de 30.
- **Óptimos locales:** la regla de Hill Climbing («solo acepto si mejora») lo hace rápido pero lo deja atrapado cuando ningún vecino mejora la ruta. La aceptación probabilística de Simulated Annealing le permite salir de esos óptimos locales.
- **Costo computacional:** ambos hacen el mismo número de iteraciones y cada una evalúa el costo completo de la ruta, O(n); en total O(iteraciones × n). Con una evaluación incremental del 2-opt (solo cambian dos aristas) cada iteración costaría O(1).
- **Limitaciones:** los datos son sintéticos, la referencia de 30 puntos es el mejor costo hallado (no un óptimo demostrado) y los parámetros se probaron en pocas combinaciones. Para afirmar mejoras más finas habría que repetir con más instancias y más corridas.

## 7. Código en Python (comentado)

Dependencias: `numpy` y `matplotlib`. Ejecución: `python FIA_6171_Sanvicente_S3.py` (imprime los resultados y genera `convergencia.png`, `rutas.png`, `cajas.png` y `puntos.png`; tarda cerca de 40 segundos).

```
"""
Actividad Parcial 2 - Sección 3: Búsqueda local y optimización
Alumno: Fernando Yair Sanvicente Valencia | Grupo 171
Asignatura: Fundamentos de Inteligencia Artificial (UP Atlautla)

Problema: ruta mínima de reparto (TSP simétrico) con 30 puntos.
Se implementan y comparan:
  - Hill Climbing (ascenso de colina, aceptación solo de mejoras)
  - Simulated Annealing (recocido simulado, enfriamiento geométrico)
Ambos usan la MISMA vecindad (2-opt), el MISMO estado inicial por semilla
y el MISMO presupuesto de evaluaciones, para que la comparación sea justa.
Como validación, en una versión pequeña de 15 puntos se calcula el óptimo
exacto con programación dinámica (Held-Karp) y se compara contra ambos algoritmos.
"""

import math, random, time, itertools, statistics
import numpy as np
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt


# ---------------------------------------------------------------------
# 1. DATOS DEL PROBLEMA (coordenadas sintéticas reproducibles, plano 100x100)
# ---------------------------------------------------------------------
def generar_instancia(n, semilla=2026):
    """Crea n puntos aleatorios reproducibles y la matriz de distancias."""
    global N_CIUDADES, CIUDADES, DIST
    N_CIUDADES = n
    rng_datos = random.Random(semilla)
    CIUDADES = [(rng_datos.uniform(0, 100), rng_datos.uniform(0, 100)) for _ in range(n)]
    DIST = [[math.dist(a, b) for b in CIUDADES] for a in CIUDADES]


generar_instancia(30)


# ---------------------------------------------------------------------
# 2. FUNCIÓN OBJETIVO Y VECINDAD
# ---------------------------------------------------------------------
def costo(ruta):
```

```
    """Función objetivo: longitud total del circuito cerrado (a minimizar)."""
    return sum(DIST[ruta[i]][ruta[(i + 1) % len(ruta)]] for i in range(len(ruta)))


def vecino_2opt(ruta, rng):
    """Vecindad 2-opt: se invierte un tramo [i, j] de la ruta.
    Reemplaza dos aristas por otras dos y siempre produce una ruta válida."""
    i, j = sorted(rng.sample(range(len(ruta)), 2))
    return ruta[:i] + ruta[i : j + 1][::-1] + ruta[j + 1 :]


# ---------------------------------------------------------------------
# 3. ALGORITMOS
# ---------------------------------------------------------------------
def hill_climbing(inicial, iteraciones, rng):
    """Hill Climbing estocástico: genera un vecino al azar y lo acepta
    solo si mejora (costo menor). Se detiene en un óptimo local."""
    actual, c_act = inicial[:], costo(inicial)
    historial = [c_act]
    for _ in range(iteraciones):
        cand = vecino_2opt(actual, rng)
        c_cand = costo(cand)
        if c_cand < c_act:
            actual, c_act = cand, c_cand
        historial.append(c_act)
    return actual, c_act, historial


def simulated_annealing(inicial, iteraciones, rng, T0=50.0, eta=0.9995):
    """Simulated Annealing: acepta empeoramientos con probabilidad
    exp(-delta/T). La temperatura baja geométricamente: T <- eta * T."""
    actual, c_act = inicial[:], costo(inicial)
    mejor, c_mejor = actual[:], c_act
    T = T0
    historial = [c_mejor]
    for _ in range(iteraciones):
        cand = vecino_2opt(actual, rng)
        c_cand = costo(cand)
        delta = c_cand - c_act
        if delta < 0 or rng.random() < math.exp(-delta / T):
            actual, c_act = cand, c_cand
            if c_act < c_mejor:
                mejor, c_mejor = actual[:], c_act
        T *= eta
        historial.append(c_mejor)
    return mejor, c_mejor, historial


def optimo_exacto():
    """Held-Karp (programación dinámica): costo mínimo exacto del TSP."""
    n = N_CIUDADES
    INF = float("inf")
    dp = {}
    for k in range(1, n):
        dp[(1 << k, k)] = DIST[0][k]
    for tam in range(2, n):
        for sub in itertools.combinations(range(1, n), tam):
            mask = sum(1 << k for k in sub)
            for k in sub:
                prev = mask & ~(1 << k)
                dp[(mask, k)] = min(dp[(prev, m)] + DIST[m][k] for m in sub if m != k)
    full = sum(1 << k for k in range(1, n))
    return min(dp[(full, k)] + DIST[k][0] for k in range(1, n))


# ---------------------------------------------------------------------
# 4. EXPERIMENTO: 30 corridas independientes por algoritmo
# ---------------------------------------------------------------------
ITER, CORRIDAS = 20000, 30


def experimento(T0=50.0, eta=0.9995):
    hc, sa = [], []
    for semilla in range(CORRIDAS):
        rng0 = random.Random(semilla)
        inicial = list(range(N_CIUDADES))
        rng0.shuffle(inicial)
        r1 = hill_climbing(inicial, ITER, random.Random(1000 + semilla))
        r2 = simulated_annealing(inicial, ITER, random.Random(2000 + semilla), T0, eta)
        hc.append(r1)
        sa.append(r2)
    return hc, sa


def resumen(nombre, res, optimo):
```

```
    """Imprime mejor, media, desviación, peor, brecha vs referencia y veces que se igualó."""
    cs = [r[1] for r in res]
    gap = 100 * (statistics.mean(cs) - optimo) / optimo
    hits = sum(1 for c in cs if c <= optimo * 1.0001)
    print(
        f"{nombre:20s} mejor={min(cs):7.2f}  media={statistics.mean(cs):7.2f}  "
        f"desv={statistics.pstdev(cs):5.2f}  peor={max(cs):7.2f}  "
        f"brecha media={gap:5.2f}%  igualó la referencia: {hits}/{len(cs)}"
    )
    return cs


def dibujar_ruta(ax, ruta, titulo):
    xs = [CIUDADES[i][0] for i in ruta + [ruta[0]]]
    ys = [CIUDADES[i][1] for i in ruta + [ruta[0]]]
    ax.plot(xs, ys, "-o", color="#1F3864", lw=1.6, ms=6)
    for pos, i in enumerate(ruta):
        ax.annotate(str(i), CIUDADES[i], textcoords="offset points", xytext=(4, 4), fontsize=7)
    ax.set_title(titulo, fontsize=9)
    ax.set_aspect("equal")
    ax.grid(alpha=0.25)


def vecino_mas_cercano():
    ruta, libres = [0], set(range(1, N_CIUDADES))
    while libres:
        s_ = min(libres, key=lambda j: DIST[ruta[-1]][j])
        ruta.append(s_)
        libres.remove(s_)
    return ruta


if __name__ == "__main__":
    t0 = time.time()
    # ---------- A) Validación en una instancia pequeña (n = 15) ----------
    generar_instancia(15, 2026)
    opt15 = optimo_exacto()
    print(f"[Validación n=15] Óptimo exacto (Held-Karp): {opt15:.2f}")
    hc15, sa15 = experimento()
    resumen("HC  (n=15)", hc15, opt15)
    resumen("SA  (n=15)", sa15, opt15)

    # ---------- B) Instancia principal (n = 30) ----------
    generar_instancia(30, 2026)
    ruta_nn = vecino_mas_cercano()
    print(f"\n[Principal n=30] Heurística del vecino más cercano: {costo(ruta_nn):.2f}")
    hc, sa = experimento()
    # Sin óptimo exacto: se usa como referencia el mejor costo hallado en todas las corridas
    sens, todos = [], [r[1] for r in hc] + [r[1] for r in sa]
    for T0, eta in [(10, 0.9995), (50, 0.9995), (200, 0.9995), (50, 0.999), (50, 0.9999)]:
        if (T0, eta) == (50, 0.9995):
            s_ = sa  # ya calculado arriba
        else:
            _, s_ = experimento(T0, eta)
        cs = [r[1] for r in s_]
        todos += cs
        sens.append((T0, eta, T0 * eta**ITER, statistics.mean(cs), min(cs), statistics.pstdev(cs)))
    ref = min(todos)
    print(f"Mejor costo conocido (referencia) = {ref:.2f}")
    print(f"\nResultados tras {CORRIDAS} corridas de {ITER} iteraciones:")
    c_hc = resumen("Hill Climbing", hc, ref)
    c_sa = resumen("Simulated Annealing", sa, ref)
    gana_sa = sum(1 for a, b in zip(c_sa, c_hc) if a < b)
    empates = sum(1 for a, b in zip(c_sa, c_hc) if a == b)
    print(
        f"Comparación pareada (mismo estado inicial): SA mejor en {gana_sa}/{CORRIDAS}, "
        f"empates {empates}, HC mejor en {CORRIDAS - gana_sa - empates}"
    )
    print("\nSensibilidad de Simulated Annealing (T0, eta):")
    for T0, eta, Tf, med, mej, sd in sens:
        print(
            f"T0={T0:5} eta={eta:<7} T_final={Tf:9.4f}  media={med:7.2f}  mejor={mej:7.2f}  desv={sd:5.2f}"
        )

    # --- Figuras ---
    fig, ax = plt.subplots(figsize=(8, 4.6))
    for h in hc[:8]:
        ax.plot(h[2], color="#C0392B", alpha=0.18, lw=0.8)
    for h in sa[:8]:
        ax.plot(h[2], color="#2E86C1", alpha=0.18, lw=0.8)
    ax.plot(
        np.mean([h[2] for h in hc], axis=0),
        color="#C0392B",
        lw=2.4,
        label="Hill Climbing (media de 30 corridas)",
```

```
)
ax.plot(
    np.mean([h[2] for h in sa], axis=0),
    color="#2E86C1",
    lw=2.4,
    label="Simulated Annealing (media de 30 corridas)",
)
ax.axhline(ref, color="#27AE60", ls="--", lw=1.5, label=f"Mejor costo conocido = {ref:.1f}")
ax.set_xlabel("Iteración (evaluaciones de vecinos)")
ax.set_ylabel("Mejor costo encontrado")
ax.set_title("Convergencia de Hill Climbing vs Simulated Annealing (30 puntos)")
ax.set_xscale("symlog", linthresh=100)
ax.set_xlim(0, ITER)
ax.legend()
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("convergencia.png", dpi=170)
plt.close(fig)

fig, axs = plt.subplots(1, 3, figsize=(11, 3.9))
dibujar_ruta(
    axs[0],
    hc[min(range(CORRIDAS), key=lambda i: hc[i][1])][0],
    f"Mejor Hill Climbing ({min(c_hc):.1f})",
)
dibujar_ruta(
    axs[1],
    sa[min(range(CORRIDAS), key=lambda i: sa[i][1])][0],
    f"Mejor Simulated Annealing ({min(c_sa):.1f})",
)
dibujar_ruta(axs[2], list(ruta_nn), f"Vecino más cercano ({costo(ruta_nn):.1f})")
fig.tight_layout()
fig.savefig("rutas.png", dpi=170)
plt.close(fig)

fig, ax = plt.subplots(figsize=(6, 4))
bp = ax.boxplot([c_hc, c_sa], tick_labels=["Hill Climbing", "Simulated\nAnnealing"], patch_artist=True)
for p, c in zip(bp["boxes"], ["#F1948A", "#85C1E9"]):
    p.set_facecolor(c)
ax.axhline(ref, color="#27AE60", ls="--", label=f"Mejor conocido = {ref:.1f}")
ax.set_ylabel("Costo final")
ax.set_title("Distribución del costo final (30 corridas)")
ax.legend()
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("cajas.png", dpi=170)
plt.close(fig)

fig, ax = plt.subplots(figsize=(4.8, 4.6))
xs, ys = zip(*CIUDADES)
ax.scatter(xs, ys, c="#1F3864", s=45)
for i, (x, y) in enumerate(CIUDADES):
    ax.annotate(str(i), (x, y), textcoords="offset points", xytext=(4, 4), fontsize=8)
ax.set_title("Los 30 puntos de reparto (coordenadas sintéticas)", fontsize=9)
ax.set_aspect("equal")
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("puntos.png", dpi=170)
plt.close(fig)
print(f"\nTiempo total de ejecución: {time.time() - t0:.1f} s")
```

## 8. Conclusiones

- Hill Climbing y Simulated Annealing resuelven el mismo problema con la misma vecindad; lo que cambia es la regla de aceptación, y esa diferencia afecta directamente la calidad de la solución.
- Hill Climbing es simple y rápido, pero depende del estado inicial y se queda en óptimos locales. Simulated Annealing tarda más en converger y a cambio da mejores resultados promedio y más consistentes.
- Los parámetros importan: un enfriamiento demasiado lento para el presupuesto de iteraciones ($\eta = 0.9999$) dio los peores resultados de Simulated Annealing.
- La instancia de validación con óptimo exacto permitió comprobar que la implementación es correcta antes de interpretar la instancia grande.
- En la siguiente etapa del curso, los algoritmos genéticos (Sección 6) comparten la idea de combinar exploración y explotación que aparece aquí.

## Referencias (APA 7.ª ed.)


Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671-680. https://doi.org/10.1126/science.220.4598.671

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.


| Declaración de uso de inteligencia artificial | |
|---|---|
| **Herramienta** | Claude (Anthropic), mediante chat web. Fecha de uso: 18 de octubre de 2026. |
| **Contexto** | Se adjuntó la hoja oficial de la Actividad Parcial 2 (imagen) para que la IA conociera los requisitos de la Sección 3: implementar Hill Climbing y Simulated Annealing, con función objetivo, vecindad, parámetros, gráfica de convergencia y comparación. |
| **Prompts enviados** | (1) Con la hoja adjunta: «Checa esto, necesito hacer todo eso, ayúdame desde cero». (2) «Ayúdame con la solicitud 3, sin la solicitud de moverle al prompt». Transcripción con ortografía corregida. |
| **Qué hizo la IA** | Propuso el problema (ruta mínima de reparto), escribió el código en Python, ejecutó el experimento, generó las gráficas, redactó el análisis y armó el PDF. |
| **Verificación realizada** | El código se ejecutó y todas las cifras del reporte provienen de esa ejecución (semillas fijas, reproducible). La implementación se validó contra el óptimo exacto (Held-Karp) en una instancia de 15 puntos. La referencia de 30 puntos es el mejor costo hallado, no un óptimo demostrado. |
| **Alcance y límites** | La IA no sustituye el criterio del alumno: las conclusiones y la entrega final son responsabilidad de quien firma este trabajo. |